



Vizualizor of climatic behavior simulation

Bridging the Visualization Gap

A 4D Interactive Atmospheric Visualizer

Team: Rafael Kipershlak & Belal Gadban [Code: 26-2-D-19]

Advisors: Dr. Zakharia Frenkel

Part 1: The Problem

Understanding the limitations of current climate modeling platforms and the need for workstation-ready 4D exploration.

The Visualization Gap

Climate models produce massive, accurate planetary data, but visualizing it remains broken.

- **Scale Conflict:** Current geospatial platforms cannot handle millions of dynamic particles.
- **The Supercomputing Bottleneck:** Rendering requires expensive HPC clusters, blocking students.
- **Engagement Gap:** 2D charts and static snapshots are unapproachable and lack 4D interactivity.

Proposed Solution & Goals



High-Fidelity

Simultaneous rendering of millions of discrete particle trajectories in real-time, visualizing atmospheric behavior.



4D Navigation

Implementing a fluid "Zoom and Scrub" interface, allowing users to navigate altitudes and temporal sequences.



Accessibility

A workstation-ready tool removing the supercomputing dependency, promoting scientific discovery through play.

Innovation vs Market

Traditional Approaches

- **Legacy APIs:** OpenGL frameworks (VisPy) cause CPU-GPU synchronization bottlenecks.
- **Game Engines:** Unity/Unreal optimize for visual approximation, losing representational accuracy.
- **Static Platforms:** Optimized only for surface mapping, not volumetric particle flow.

Our Custom Architecture

- **Low-Level API:** Direct hardware control using Vulkan to bypass background drivers.
- **Absolute Fidelity:** Rendering remains completely faithful to the raw ingested simulation datasets.
- **Democratization:** Achieves high-fidelity scientific visualization on consumer-grade hardware, bypassing the need for supercomputing clusters.

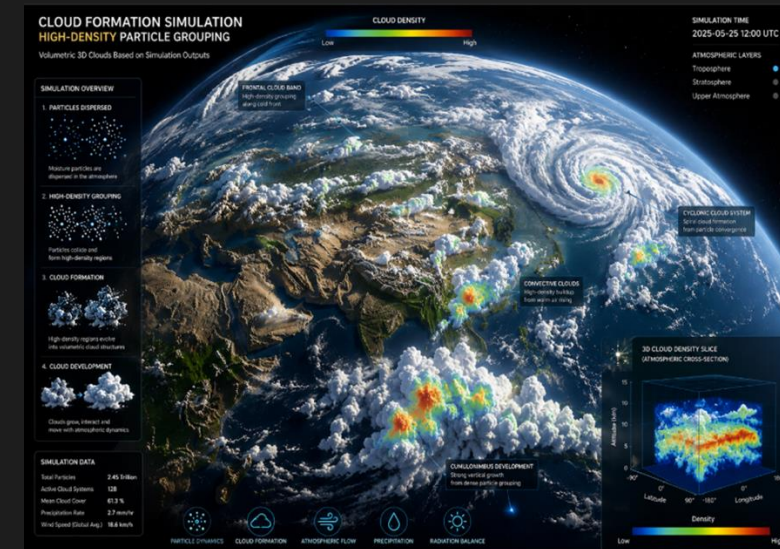
Innovation vs Market - Example

Traditional Approaches



Cloud Formation visualization – today (CesiumJS)

Our Custom Architecture



Why Vulkan?

What it is?

- Low-level GPU API
- Direct hardware control
- Minimal driver overhead

Why choose Vulkan?

- Eliminates CPU-GPU bottlenecks
- Enables massive parallelism
- Required for real-time millions of particles

Tradeoff

- Complex to implement
- Manual memory & synchronization management

Part 2: Engineering

Addressing I/O bottlenecks, asynchronous pipelines, and GPU compute shaders.

Core Research Challenges

I/O

JSON Bottleneck

Parsing JSON frames causes CPU stalling and ruins real-time FPS rendering constraints.

PCIe

Bandwidth Limit

Frequent CPU to GPU transfers of millions of particles completely saturates PCIe bandwidth.

4D

Temporal Aliasing

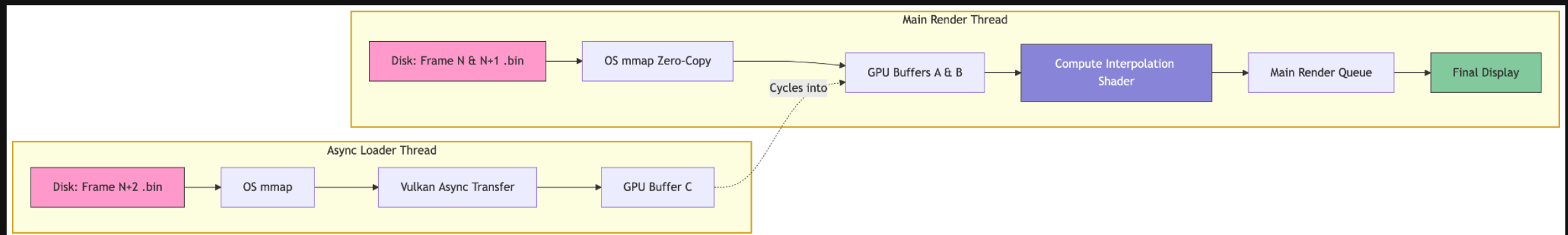
Direct discrete frame-to-frame rendering produces jumping, discontinuous atmospheric motion.

The Vulkan Advantage

Asynchronous Streaming Architecture

To solve the I/O and PCIe bottlenecks, we engineered a triple-buffered streaming pipeline.

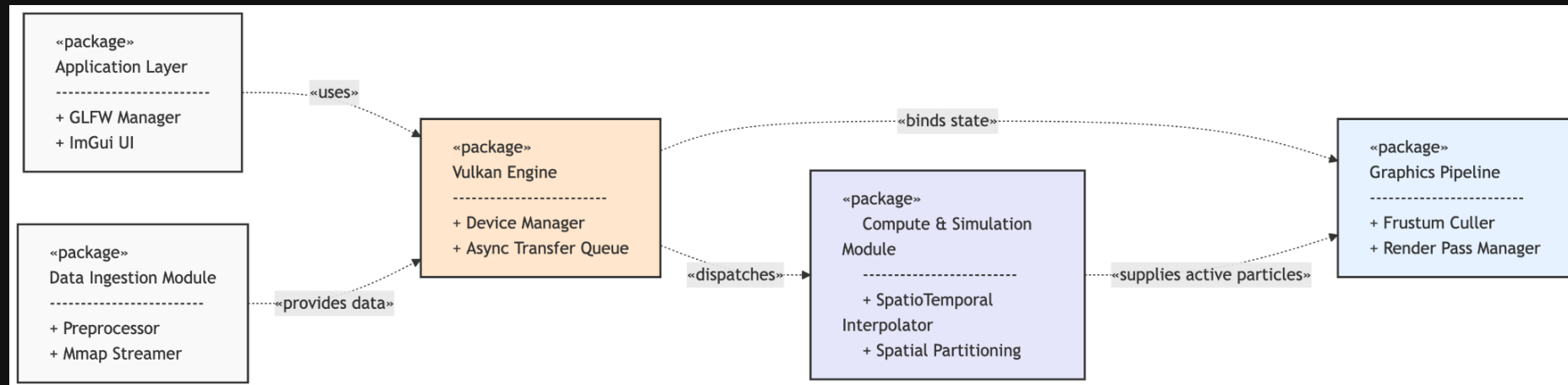
- **Offline Preprocessing:** JSON converted to contiguous binary files beforehand.
- **Memory Mapping (mmap):** Zero-copy file mapping bypasses OS parsing overhead.
- **Decoupled Pipeline:** Async loader pushes Frame N+2 to VRAM while main thread renders Frame N.



System Architecture

Modular Separation of Concerns

- **Vulkan Engine:** Central orchestration, managing hardware lifecycle and resource allocation.
- **Data Ingestion:** Independent mmap streaming.
- **Compute Module:** Parallel shaders for spatial partitioning and interpolation.
- **Graphics Pipeline:** Visibility filtering and final frame draw presentation.



GPU Spatio-Temporal Interpolation

To eliminate temporal aliasing, Lagrangian interpolation is calculated entirely on the GPU compute shader for millions of particles simultaneously.

These equations compute particle positions between frames, eliminating visual discontinuities.

Temporal Factor

$$\alpha = \frac{t - t_N}{t_{N+1} - t_N}$$

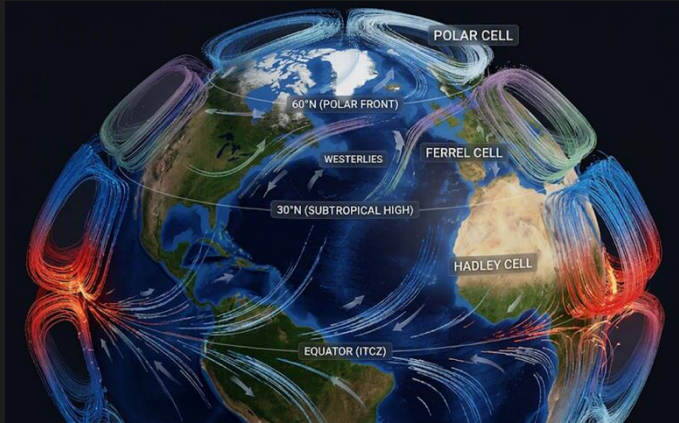
Position Integration

$$P = (1 - \alpha) P_N + \alpha P_{N+1}$$

Visibility Optimization Filters:

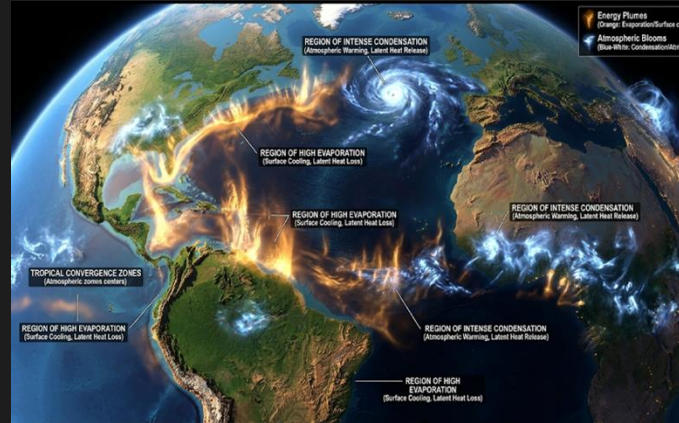
- **Spatial Partitioning:** Grid/Octree bounding.
- **Frustum Culling:** Discard out-of-camera particles.
- **LOD:** Distance-based density sampling

Visualizing Atmospheric Data



Convective Cells

Macro-structures (Hadley, Ferrel) mapped using looping vector streamlines.



Latent Heat Fluxes

Energy hotspots highlighting phase transitions from vapor to liquid.



Condensation Zones

Volumetric density markers driven by pressure and temperature gradients.

User Interface Design

Scientific Discovery through Play

- **Control Panel:** Granular toggling of atmospheric layers (Heat, Vapor, Wind).
- **3D Viewport:** The main GPU rendering window for real-time interaction.
- **Timeline Scrubbing:** Fluid temporal navigation forwards/backwards.
- **Live Analytics:** On-the-fly histograms and velocity charts based on current view.



QA & Testing Plan

Test Category	Objective / Scenario	Pass Criteria
Performance (PT-01)	Validate real-time rendering across massive workloads (500k - 5M particles).	Minimum 30 FPS under maximum 5M particle load.
Latency (PT-02)	Measure interaction delay during timeline scrubbing and zoom.	System response time is ≤ 100 ms.
Visual Integrity (VT-01)	Compare rendered color of coordinates against raw numeric values.	Perfect alignment with designated scientific colormap.
Stress Testing	Extreme load (10M particles), rapid dataset switching.	Zero crashes, automatic LOD activation verified.

Questions?

Thank you for your attention.

4D Interactive Atmospheric Visualizer

Braude Academic College